cDFT.jl

Pierre J. Walker and Andrés Riedemann.

February 23, 2026

Contents

Part I

Home

Chapter 1

cDFT.jl

cDFT intends to provide a comprehensive library of classical Density Functional Theory and Self-Consistent Field-Theory models, as well as a simple framework to develop your own!

The documentation is laid out as follows:

- **API**: A list of all available methods.

Authors

- Pierre J. Walker, California Institute of Technology
- Andrés Riedemann, University of Concepción

Part II

API

Chapter 2

System

Contents

System type

cDFT.DFTSystem – Type.

```
DFTSystem(model::EoSModel, species::DFTSpecies, structure::DFTStructure,
↪  fields::Vector{DFTField}, options::DFTOptions)
```

Generic struct which includes all the information needed to perform DFT / SCFT calculations:

- `model`: A model object that should be obtained from Clapeyron.jl, and contains all information regarding species parameters.
- `species`: A DFTSpecies object which is model-dependent. Typically contains the number of beads in each species and the bead sizes.
- `structure`: A DFTStructure object which provides information regarding the geometry (including bounds) and conditions (n, p, T) of the DFT calculations.
- `fields`: A vector of DFTFields for each field used in the DFT-calculation. This is typically model-dependent.
- `options`: A DFTOptions object which contains information regarding the convergence settings and the devices used as part of the DFT calculation.

Example usage:

```
julia> model = PCSAFT(["water"])

julia> L = length_scale(model)

julia> structure = Uniform1DCart((1e5, 298.15, [1.]), [0, 20L], 201)

julia> system = DFTSystem(model, structure)
DFTSystem
  model: PCSAFT{BasicIdeal, Float64}
         with 1 component: "water"
  structure: Uniform1DCart
  device: CPU
```

source

Chapter 3

Methods

Contents

Solvers

`cDFT.converge!` – Function.

```
converge!(system::DFTSystem, ρ)
```

For a given system, converge the profiles using the solver specified under `system.options.solver`. Convergence is achieved by solving the generic equation:

```
ρi = ρi_bulk*exp(β(μi_res - δFδρ_res))
```

For stability purposes, the equation has be reformulated as:

```
ln(ρi) = ln(ρi_bulk) + β(μi_res - δFδρ_res)
```

The default solver uses Anderson Mixing with 100 initial Picard iterations, 50 memory points, 1e-2 damping, and an infinite drop tolerance.

source

Properties

`cDFT.surface_tension` – Function.

```
surface_tension(model::EoSModel,T,x)
```

Calculate the surface tensions of a vapour-liquid interface based on a given `model` and at saturated conditions T and x (liquid composition). This is a blind calculation that assumes the interface is 1D and cartesian, and that the system does phase split between two bulk phases. If the system is expected to form micelles, other methods should be used.

Example:

```
julia> model = PCSAFT(["water"])

julia> surface_tension(model,298.15,[1.0])
```

source

`cDFT.interfacial_tension` – Function.

```
interfacial_tension(model::EoSModel,p,T,x,xx)
```

Calculate the interfacial tension of a liquid-liquid interface based on a given `model` and at conditions p, T, x and xx, where x and xx are the compositions of the two phases, respectively. This calculation assumes the interface is 1D and cartesian, and that the two bulk phases are in equilbrium. If the system is expected to form micelles, other methods should be used.

Example:

```
julia> model = PCSAFT(["water","hexane"])

julia> (n,_,_) = tp_flash(model,1e5,298.15,[0.5,0.5],RRTPFlash(equilibrium=:lle))

julia> interfacial_tension(model,1e5,298.15,n[1,:],n[2,:])
```

source

```
interfacial_tension(system::DFTSystem, ρ)
```

Calculate the interfacial tension of an based on a given `system` and density profile ρ. To get accurate results, the system should be converged first.

Example:

```
julia> model = PCSAFT(["water","hexane"])

julia> (n,_,_) = tp_flash(model,1e5,298.15,[0.5,0.5],RRTPFlash(equilibrium=:lle))

julia> v1 = volume(model,1e5,298.15,n[1,:])

julia> v2 = volume(model,1e5,298.15,n[2,:])

julia> ρ1 = n[1,:]./v1

julia> ρ2 = n[2,:]./v2

julia> structure = TwoPhase1DCart((1e5, 298.15), ρ1, ρ2, [-10L, 10L], 201)

julia> system = DFTSystem(model, structure)

julia> ρ = initialize_profiles(system)

julia> converge!(system, ρ)

julia> interfacial_tension(system, ρ)
```

source

Chapter 4

Structure

Contents

Types

cDFT.Uniform1DCart – Type.

```
Uniform1DCart(conditions::Tuple{Float64,Float64}, ρbulk, bounds::Vector{Float64}, ngrid::Int64)
```

The generic structure type used when trying to simulate a uniform system in 1D-cartesian coordinates. Contains:

- `conditions`: The p, T conditions of the system.
- `ρbulk`: The bulk density of each species in the system.
- `bounds`: Specifies the location of the bounds of the system.
- `ngrids`: The number of grid points used to represent the density profile.

This structure should generally be used to benchmark the DFT code against the bulk calculations. Example:

```
julia> structure = Uniform1DCart((p, T) , ρbulk, [-10L, 10L], 201)
```

source

cDFT.TwoPhase1DCart – Type.

```
TwoPhase1DCart(conditions::Tuple{Float64,Float64}, ρbulk, ρbulk2, bounds::Vector{Float64},
↪  ngrid::Int64)
```

The generic structure type used when trying to simulate two-phase interfaces in 1D-cartesian coordinates. Contains:

- `conditions`: The p, T conditions at which the calculations are performed.
- `ρbulk`: The bulk density of each species in the first phase.
- `ρbulk2`: The bulk density of each species in the second phase.
- `bounds`: Specifies the location of the bounds of the system. The interface will be located in the middle.
- `ngrids`: The number of grid points used to represent the density profile.

In the case of the surface tension calculation, the pressure specified must be the saturation pressure at T and x. As the density profiles in the liquid and vapour phases are expected to reach their bulk values, the densities at the bounds are *fixed* at their bulk values. In general, it is recommends to use a width of about 20L (L being the length scale of the model) and about 201 grid points.

The profiles will be initialised as generic sigmoidals of the form: `tanh_prof(x,start,stop,shift,coef) = 1/2*(start-stop)*tanh((x-shift)*coef)+1/2*(start+stop)`

Example:

```
julia> model = PCSAFT(["water"])

julia> T = 298.15

julia> (p, vl, vv) = saturation_pressure(model, T)

julia> ρ1 = [1.0]./vl

julia> ρ2 = [1.0]./vv

julia> L = length_scale(model)

julia> structure = TwoPhase1DCart((p, T), ρ1, ρ2, [-10L, 10L], 201)
```

source

cDFT.ExternalField1DCart – Type.

```
ExternalField1DCart(conditions::Tuple{Float64,Float64}, ρbulk::Vector{Float64},
↪  bounds::Vector{Float64}, ngrid::Int64, external_field::ExternalFieldModel, width::Float64)
```

The generic structure type used when trying to simulate solid-fluid interfaces in 1D-cartesian coordinates. Contains:

- `conditions`: The p, T conditions of the total system before it splits between the two liquid phases.
- `ρbulk`: The bulk density of each species in the system.
- `bounds`: Specifies the location of the bounds of the system. The interface will be located in the middle.
- `ngrids`: The number of grid points used to represent the density profile.
- `external_field`: The external field model used to calculate the external field.
- `width`: The surface-to-surface separation.

Example:

```
julia> model = PCSAFT(["carbon dioxide"])

julia> ρbulk = [molar_density(model,1e5,298.15)]

julia> L = length_scale(model)

julia> H = 10L

julia> surface = Steele(["graphite"])

julia> structure = InterfacialTension1DCart((p, T), ρbulk, [0.5L, H-0.5L], 201, surface, H)
```

source

cDFT.Uniform1DSphr – Type.

```
Uniform1DSphr(conditions::Tuple{Float64,Float64}, ρbulk::Vector{Float64},
↪  bounds::Vector{Float64}, ngrid::Int64)
```

The generic structure type used when trying to simulate a uniform system in 1D-spherical coordinates. Contains:

- `conditions`: The p, T conditions of the system.
- `ρbulk`: The bulk density of each species in the system.
- `bounds`: Specifies the location of the bounds of the system.
- `ngrids`: The number of grid points used to represent the density profile.

This structure should generally be used to benchmark the DFT code against the bulk calculations. As this is spherical coordinates, the lower bound must be greater than 0. Example:

```
julia> structure = Uniform1DSphr((p, T), ρbulk, [0L, 10L], 201)
```

source

Functions

cDFT.initialize_profiles – Function.

```
initialize_profiles(system::DFTSystem)
```

Based on the system specifications, this function will initialize the density profiles for each of the species / beads in the model. The output will be an array of size `(ngrid,nb)` where `ngrid` is the number of grid points used and `nb` is the number of beads.

Example:

```
julia> model = PCSAFT(["water"])

julia> ρbulk = [molar_density(model,1e5,298.15)]

julia> L = length_scale(model)

julia> structure = Uniform1DCart((1e5, 298.15), ρbulk, [0, 20L], 201)

julia> system = DFTSystem(model, structure)

julia> profiles = initialize_profiles(system)
```

source

Chapter 5

Fields

Contents

Types

cDFT.SWeightedDensity – Type.

```
SWeightedDensity(type::Symbol,width::Vector{Float64},map::Array{ComplexF64})
```

Generic SWeightedDensity type used to calculate the scalar weighted densities of the system. One must specify:

- type: The type of weighted density to be calculated. Options are :∫ρdz (n_0), :∫ρzdz (n_v), :∫ρz²dz (n_3), and :ρ (unweighted).
- width: The width of the weighted density profile.
- map: The Fourier transform of the weights.
- plan: The Fourier transform plan.
- iplan: The inverse Fourier transform plan.

source

cDFT.VWeightedDensity – Type.

```
VWeightedDensity(type::Symbol,width::Vector{Float64},map::Array{ComplexF64})
```

Generic VWeightedDensity type used to calculate the vector weighted densities of the system. One must specify:

- type: The type of weighted density to be calculated. Options are :∫ρdz (n_0), :∫ρzdz (n_v), :∫ρz²dz (n_3), and :ρ (unweighted).
- width: The width of the weighted density profile.
- map: The Fourier transform of the weights.
- plan: The Fourier transform plan.
- iplan: The inverse Fourier transform plan.

source

Functions

cDFT.evaluate_field – Function.

```
evaluate_field(system::DFTSystem, profiles)
```

This function will obtain every field used in the system (listed in system.fields). The output will be a 3D array with the dimensions (ngrid,nf,nc), where ngrid is the number of grid points, nf is the number of fields, and nc is the number of components in the model.

This is the macro function that will call evaluate_field(system,field,profiles) for each field in the system.

source

cDFT.integrate_field – Function.

```
integrate_field(system::DFTSystem, δf, ρ)
```

This function will obtain, for all fields, the functional derivative for each species / bead. The output will be a 2D array with the dimensions (ngrid,nb), where ngrid is the number of grid points, and nb is the number of beads in the model.

This is the macro function that will call integrate_field(system,field,δf,ρ) for each field in the system.

source

Chapter 6

Profiles

Contents

Types

Functions

Chapter 7

Utils

Contents

Chapter 8

Options

Contents

Types

cDFT.DFTOptions – Type.

```
DFTOptions(device::Device, solver::Solvers.AbstractFixPoint)
```

A struct which includes all the settings that need to be set for the convergence algorithms and devices used:

- `device`: Specification of either CPU (pinned or un-pinned) or GPU devices. (unpinned CPU by default)
- `solver`: Specification of the solver type and solver settings used. Must be a fixed-point method. (`AndersonFixPoint` by default)

Example usage:

```
julia> options = DFTOptions()

julia> using ThreadPinning

julia> options = DFTOptions(CPU(4, [0,1,12,13]))
```

source

cDFT.CPU – Type.

```
CPU(ncpu::Int, pinning::Bool, device_ids::Vector{Union{Nothing,Int}})
```

A struct containing information regarding the CPU settings.

- `ncpu`: Number of CPU threads used
- `pinning`: Specifies whether pinning is used or not.
- `device_ids`: When CPUs are pinned, specify the ID of those threads.

source